Technical Learning Report: Day 2

Date: September 2, 2026
Intern Name: Shekhar Singh Rawal
Subject: SDLC Models & Test Levels

1. Objective

The objective of Day 2 was to develop an understanding of Software Development Life Cycle (SDLC) models and the different levels of software testing associated with them. The learning focused on the Waterfall, V-Model, and Agile/Scrum approaches, including sprint cycles and the basic concept of Continuous Integration (CI). The day also focused on understanding how QA activities and different testing levels such as Unit, Integration, System, and Acceptance Testing fit within different SDLC phases. Practical activities were performed using a student-registration application to understand where each testing level is applied and what type of issues it helps identify.

2. Technical Breakdown

A. Software Development Life Cycle (SDLC)

SDLC stands for **Software Development Life Cycle**. It is a structured process used to plan, develop, test, deploy, and maintain software.

The SDLC provides an organized approach to software development by defining different stages through which a software product progresses. The understanding developed during the session focused on how different SDLC models organize these activities differently.

B. Waterfall Model

The **Waterfall model** is a sequential SDLC model in which each phase is completed before moving to the next phase.

The basic flow discussed was:

Requirements → Design → Development → Testing → Deployment

Key characteristics understood:

- Development activities follow a sequential structure.
- Each phase is completed before proceeding to the following phase.
- Testing generally takes place after the development phase.
- Acceptance testing is performed before deployment.
- The model is suitable when requirements are stable.

C. V-Model

The **V-Model** is an SDLC model in which development and testing activities are planned together. Each development phase has a corresponding testing phase.

The major understanding from the session was that testing is considered alongside development rather than being treated only as a final activity.

The V-Model was identified as useful when:

- Requirements are stable.
- Strong verification and validation are important.
- Testing activities need to be planned alongside development activities.

D. Agile/Scrum and Sprint Cycles

Agile is an iterative and flexible approach in which software is developed in small increments and tested continuously.

A **Sprint** in Agile/Scrum is a short, fixed period during which a team develops and tests a specific set of features. The provided learning material described a Sprint as usually lasting **1–4 weeks**.

The main differences understood compared with sequential models were:

- Development takes place incrementally.
- Testing occurs continuously throughout each Sprint.
- Developers and QA work together.
- Features can be developed, tested, fixed, and retested within the Sprint.
- Agile is useful when requirements may change and frequent feedback is required.

E. Test Levels

Four main levels of testing were covered:

1. Unit Testing

Testing individual components or functions of the application.

2. Integration Testing

Testing whether different components work correctly together.

3. System Testing

Testing the complete application as an integrated system.

4. Acceptance Testing

Checking whether the application satisfies the required business or user requirements.

The student-registration application was used to understand how these levels are positioned as the application progresses from individual components toward complete user requirements.

F. Continuous Integration (CI)

CI, or **Continuous Integration**, was covered as a basic concept.

The understanding developed was that developers frequently integrate their code into a shared repository, while automated builds and tests can provide quick feedback about the integrated changes.

G. Comparison of SDLC Models

The learning activity included comparing SDLC models based on project requirements and development characteristics.

The scenario analysis resulted in the following mapping:

Scenario	SDLC Model	Reason
Banking compliance app	**V-Model**	Requirements are generally stable, and strong verification and validation are important. Testing activities can be planned alongside development activities.

Startup MVP	**Agile**	An MVP may require frequent changes based on feedback. Agile supports iterative development, frequent testing, and continuous feedback.
IoT firmware update	**Waterfall / V-Model**	When firmware requirements are stable and verification is important, a sequential or V-Model approach provides a structured development and testing process.

3. Practical Implementation

A. SDLC Questions and Answers

A set of **10 questions on SDLC models** was completed. The questions covered:

- Definition of SDLC
- Waterfall model
- V-Model
- Agile
- Sprint cycles
- Four levels of testing
- Testing in Waterfall
- Testing in Agile
- Continuous Integration
- Selection of an appropriate SDLC model based on project requirements

The answers demonstrated understanding of the differences between the SDLC models and how testing activities are handled within them.

B. Student Registration Application — SDLC and Test-Level Mapping

A practical SDLC flow was created for a **Student Registration App**.

The application flow was mapped as:

1. Requirements
Students can register for courses.

↓

2. Design
UI + API + Database + Architecture

↓

3. Development
Login + Course Search + Registration

↓

Unit Testing
Testing individual functions

↓

4. Integration Testing
Checking whether UI, API, and Database work together

↓

5. System Testing
Testing the complete registration application

↓

6. Acceptance Testing
Checking the business/user requirements

↓

7. Deployment
Application released to students

This exercise helped map the different testing levels to the development flow of a real application scenario and understand their purpose at different stages.

C. SDLC Scenario-Matching Activity

A scenario-matching exercise was completed using three different project types:

- Banking compliance application
- Startup MVP
- IoT firmware update

The activity focused on selecting an appropriate SDLC model for each scenario and providing a reason for the selection.

The analysis showed that the choice of SDLC model depends on project characteristics rather than there being one universally best model.

D. AI-Assisted Learning

AI-assisted learning was included as part of the day's activities. The provided learning work focused on comparing Waterfall, V-Model, and Agile approaches and understanding their relevance to QA.

The resulting understanding was compared through the completed SDLC questions, model comparison, and scenario analysis rather than treating one SDLC model as universally superior.

4. Key Findings & Best Practices

- SDLC provides a structured process for planning, developing, testing, deploying, and maintaining software.
- Waterfall follows a sequential development approach, with testing generally occurring after development.
- The V-Model connects development phases with corresponding testing activities and emphasizes planning testing alongside development.
- Agile follows an iterative approach where development and testing occur continuously within Sprints.
- A Sprint is a short, fixed development period used to develop and test a specific set of features.
- Unit, Integration, System, and Acceptance Testing represent different levels of testing, from individual components through complete application and user/business requirements.
- The appropriate SDLC model depends on the characteristics and requirements of the project.
- Stable requirements and strong verification/validation requirements may favor a V-Model or Waterfall-style approach.
- Projects such as startup MVPs that may require frequent changes and feedback can benefit from an Agile approach.
- Testing in Agile is not treated only as a final development activity; it occurs continuously throughout the Sprint.
- CI provides quick feedback by integrating code into a shared repository and using automated builds and tests.

5. Conclusion

Day 2 focused on understanding SDLC models and their relationship with software testing. The Waterfall, V-Model, and Agile/Scrum approaches were studied along with Sprint cycles and the basic concept of Continuous Integration. The four major test levels—Unit, Integration, System, and Acceptance Testing—were also studied and mapped to a student-registration application. Practical learning was demonstrated through the completion of 10 SDLC questions, creation of the student-registration SDLC and test-level flow, and scenario-based selection of SDLC models for banking, startup MVP, and IoT projects. Overall, the day's activities strengthened the understanding of how development approaches influence the timing and placement of QA and testing activities.